

IndicLex

Multilingual Dictionary Management & Search Platform

Project Documentation

CS 325: Web Application Development | Iterations 1 – 10

Course	CS 325: Web Application Development
Project	IndicLex
Platform	PHP + MySQL on Bluehost
Iterations	10 (all complete)
Final Status	Complete — Deployed on Bluehost
Group Name	Cougars
Group Members	Brian Williams Casey Cunningham Jimmy Taiwo Abdi Osman Salmaan Sahal

Project Overview

IndicLex is a full-stack web application built for CS 325: Web Application Development. The platform provides end-to-end management and search for multilingual dictionaries, with a focus on Indic languages including Telugu, Sanskrit, and Hindi. The project was developed over 10 sprints and deployed on Bluehost.

The application supports bilingual and trilingual dictionaries. Users can upload word lists from Excel files, search with four different modes, manage preferences via cookies, and access dictionary data through a REST API. Administrators can manage all content through a secure admin panel.

Pages & Files

index.php	Home — Excel upload form, feature cards, upload modal with full validation
catalog.php	Dictionary Catalog — create/delete dicts, view entries, summary stats
search.php	Search — 4 modes, paginated results, jQuery autocomplete, Word Length Matcher
preferences.php	Preferences — theme, results per page, default dict/mode; full cookie management
api_search.php	REST API — GET /api/search, JSON responses, correct HTTP status codes
dbcon.php	Database connection via mysqli
.htaccess	Apache config — URL rewriting, security headers, gzip, caching, file blocking
js/theme.js	Theme toggle — reads/writes cookie, applies before render to prevent flash
css/indiclex-custom.css	Custom CSS — light/dark theme via CSS variables, sidebar, cards, badges
indiclex_schema_ics325.sql	Full schema — triggers, views, indexes, seed data

Technology Stack

Backend	PHP — form handling, session management, API responses, file processing
Database	MySQL (InnoDB, utf8mb4) — normalized schema, foreign keys, triggers
Frontend	Bootstrap 5.3, Font Awesome 6.5, custom CSS variables for theming
JavaScript	jQuery 3.7 — autocomplete, AJAX, Word Length Matcher, sidebar toggle
File Handling	PhpSpreadsheet (Composer) — reads .xlsx files for bulk dictionary import

Hosting	Bluehost shared hosting — Apache with mod_rewrite, .htaccess configuration
Version Control	GitHub

Database Schema

The schema defines four tables with normalization, FK constraints, auto-update triggers, and two reporting views.

dictionaries	Dictionary metadata: name, type (bilingual/trilingual), source languages, entry_count, is_active, timestamps
dictionary_entries	Word entries: lang_1/lang_2/lang_3, pronunciation, part_of_speech, notes, example. UNIQUE on (dict_id, lang_1). Indexed on all three lang columns plus FULLTEXT.
users	Admin accounts with bcrypt password hashes and role ENUM (admin/visitor)
preferences	System-wide defaults (theme, results_per_page, default_mode) used as fallback in the cookie resolution chain

Triggers: trg_entry_insert and trg_entry_delete keep dictionaries.entry_count in sync automatically on every insert or delete.

Views: vw_dictionary_stats and vw_language_stats provide pre-aggregated data for the statistics and reporting pages.

Sprint Summary

#	Title	Key Deliverables	Status
1	Project Setup & Database Foundation	DB schema, Bluehost setup, PHP connection layer	Complete
2	Public UI Shell & Navigation	Bootstrap sidebar layout, theme toggle, Dictionary Catalog page	Complete
3	Bulk Import/Export & File Handling	Excel upload via PhpSpreadsheet, duplicate detection, error reporting	Complete
4	Search Engine (Core)	4 search modes, parameterized queries, paginated results	Complete
5	Preferences & Cookie Management	Cookie resolution chain, preferences page, save/reset	Complete
6	Admin Authentication & Dashboard	Login, bcrypt, sessions, route protection, dashboard stats	Complete
7	Dictionary & Entry CRUD	Create/delete dicts and entries, cascade deletes, modal UI	Complete
8	REST API & Advanced Search	GET /api/search, autocomplete AJAX, Word Length Matcher	Complete
9	Comparison, Data Integrity & Reporting	DB views, stats reporting, integrity enforcement via schema	Complete
10	Stabilization, Polish & Demonstration	.htaccess hardening, UI polish, end-to-end testing, final demo	Complete

Iteration 1 — Project Setup & Database Foundation

Status: Complete

What Was Planned

- Set up GitHub repository and Bluehost hosting
- Design and implement the normalized MySQL schema
- Establish the PHP database connection layer
- Deploy project skeleton and confirm live URL

What Was Built

- MySQL schema created with four tables: dictionaries, dictionary_entries, users, and preferences — all using InnoDB and utf8mb4 for full Unicode/Indic script support
- dictionaries table supports bilingual and trilingual types via a type ENUM column with an optional source_lang_3 field
- dictionary_entries uses lang_1/lang_2/lang_3 columns with a UNIQUE constraint on (dict_id, lang_1) to prevent duplicate headwords at the database level
- Triggers trg_entry_insert and trg_entry_delete automatically keep the entry_count column on dictionaries in sync with every insert and delete
- Two reporting views created: vw_dictionary_stats and vw_language_stats, for use in the statistics and reporting pages
- FULLTEXT index on (lang_1, lang_2, lang_3) added to dictionary_entries for future full-text search expansion
- dbcon.php provides a shared mysqli connection used across all pages
- Seed data inserted for sample dictionaries (Telugu–English, Sanskrit–English, English–Telugu–Hindi) and a default admin user
- Bluehost environment live and confirmed — PHP and MySQL connectivity verified through the browser

Challenges & Decisions

- Character encoding set to utf8mb4 across the connection and all tables to correctly handle Telugu, Sanskrit, Hindi, and other multi-byte Indic scripts
- Storing dictionary type (bilingual/trilingual) in the dictionaries table lets the import logic dynamically decide whether lang_3 is required without extra application logic
- Database-level UNIQUE constraints handle duplicate detection automatically so the application does not need to query for duplicates before every insert

Demo Notes

- Live Bluehost URL loads successfully with no PHP or database errors
- phpMyAdmin confirms all four tables exist with correct column definitions and sample seed data
- GitHub repository shows clean initial commit containing the schema file and project skeleton

Iteration 2 — Public UI Shell & Navigation

Status: Complete

What Was Planned

- Build a Bootstrap-based responsive layout with sidebar and top navbar
- Create the Home page with a hero section and feature cards
- Build the Dictionary Catalog page pulling live data from the database
- Implement a light/dark theme toggle using cookies

What Was Built

- Bootstrap 5.3 layout built with a sticky left sidebar (220px min-width) and a flex content wrapper — consistent structure across all four public pages
- Sidebar includes Font Awesome icons for Home, Dictionary Catalog, Search, and Preferences with active-state highlighting on the current page
- Home page hero section displays the IndicLex brand and four feature cards linking to the main sections of the app
- Dictionary Catalog page dynamically shows active dictionary count, unique source language count, and the full dictionary table from the database
- Light/dark theme toggle implemented in `js/theme.js` — reads the `indiclex_theme` cookie and sets a `data-theme` attribute on the `html` element before the page renders, eliminating any flash of unstyled content
- Theme is stored in a cookie with a 120-day expiry and drives all visual styles through CSS custom variables (`--il-surface`, `--il-text`, `--il-primary`, etc.)
- Custom CSS in `css/indiclex-custom.css` defines the full variable palette for both themes in a single stylesheet
- Mobile-friendly sidebar collapse toggle added for small screen sizes

Challenges & Decisions

- Theme JS is loaded in the `<head>` before any HTML body content — this ordering is critical to prevent a visible flash when dark mode is active on page load
- CSS custom variables mean switching themes requires only a single attribute change on the `html` element, with no JavaScript class toggling per component
- Sidebar layout chosen over a top navbar to preserve vertical content space, which matters on search and catalog pages with long result tables

Demo Notes

- Sidebar navigation works correctly across all pages with the active indicator on the current page
- Theme toggle persists across page reloads — no visible flash on dark mode
- Dictionary Catalog shows the correct live count of dictionaries and source languages from the database

Iteration 3 — Bulk Import/Export & File Handling

Status: Complete

What Was Planned

- Implement bulk dictionary upload from Excel files using PhpSpreadsheet
- Add file validation, duplicate detection, and row-level error reporting
- Handle edge cases including large files, malformed rows, and Bluehost upload limits

What Was Built

- Excel upload fully implemented on index.php — accepts .xlsx files with both extension and MIME type validation before any file is processed
- PhpSpreadsheet (loaded via Composer/vendor/autoload.php) reads each spreadsheet row and inserts entries into dictionary_entries using a prepared statement
- Upload form includes a dictionary selector — the chosen dictionary's type (bilingual/trilingual) determines whether lang_3 is required or skipped per row
- File size capped at 5 MB with a PHP-side check run before PhpSpreadsheet is invoked
- Folder management: if the uploads/ directory already contains 5 files, the oldest is auto-deleted before saving the new file — prevents disk bloat on shared hosting
- Uploaded filenames are sanitized with a regex before being written to disk to prevent path traversal issues
- Duplicate rows are caught by the database UNIQUE constraint and reported as skipped — the upload continues processing remaining rows rather than aborting
- Row-level feedback displayed after upload: rows inserted, duplicates skipped, and validation errors listed by row number
- If the upload form had validation errors on POST, the upload modal automatically re-opens on page load so the user does not lose context
- Export available through the Dictionary Catalog table and the REST API for programmatic access to dictionary data

Challenges & Decisions

- Both extension and MIME type checks were added because some systems report .xlsx as application/zip — accepting both prevents false rejections of valid files
- Continuing to process remaining rows on a duplicate or missing-field error (rather than aborting the whole upload) is much more practical for large real-world dictionary files
- The 5-file auto-delete limit is specific to Bluehost shared hosting constraints and keeps the uploads directory from accumulating stale files over time

Demo Notes

- Upload a .xlsx file with 50 valid rows — confirm success message shows correct insert count
- Upload the same file a second time — confirm duplicates are reported as skipped and no database error occurs
- Upload a file with a row missing lang_1 — confirm the row error is listed and the rest of the file processes normally
- Attempt to upload a .csv file — confirm the file type error blocks the upload

Iteration 4 — Search Engine (Core)

Status: Complete

What Was Planned

- Build the Search Interface with a dictionary selector and search mode options
- Implement parameterized MySQL queries for all four search modes
- Display paginated results with result count and navigation

What Was Built

- Search page (search.php) uses a two-column layout: search parameters on the left, results on the right
- Dictionary dropdown is populated from the database — users can search one dictionary or all active dictionaries at once
- Four search modes implemented: exact (full match), prefix (LIKE term%), suffix (LIKE %term), and substring (LIKE %term%) — all using parameterized prepared statements
- Wildcard characters % and _ in user input are escaped before being used in LIKE patterns to prevent unintended broad matches
- Results table shows headword (lang_1), translation (lang_2), pronunciation (when available), part of speech, and source dictionary name
- Pagination implemented with a configurable page size — shows current page, total pages, and total result count in the card footer
- All search parameters (query, mode, dict_id, page) are preserved in pagination URL links so results are bookmarkable and shareable
- Empty query state renders a 'Ready to Search' placeholder panel rather than querying the database
- No-results state shows a user-friendly message with a suggestion to try substring mode
- A Search Mode Reference table at the bottom explains each mode with an example query and the corresponding SQL LIKE pattern

Challenges & Decisions

- Wildcard escaping was added after testing found that a query containing % returned all entries — the fix escapes user input before building the LIKE pattern
- Searching only lang_1 (the headword) rather than all columns keeps results predictable and avoids accidentally surfacing translations as if they were headwords
- URL parameter preservation in pagination links means navigating between result pages does not reset the search form — important for usability with large result sets

Demo Notes

- Search for a known exact word — confirm one result appears
- Search with a prefix — confirm multiple results across pages and working pagination
- Enter a query containing % — confirm it is treated as a literal character
- Navigate to page 2 of a result set and verify the row numbering picks up correctly from the previous page

Iteration 5 — Preferences & Cookie Management

Status: Complete

What Was Planned

- Build the Preferences Panel for theme, default dictionary, results per page, and default search mode
- Implement the full preference resolution chain: cookie → database → hard-coded fallback
- Allow users to reset preferences to system defaults

What Was Built

- Preferences page (preferences.php) has a save/reset form on the left and a live Active Preference Sources panel on the right
- Four preferences tracked with their own cookies: indiclex_theme, indiclex_rpp, indiclex_dict, and indiclex_mode
- All incoming preference values are whitelist-validated on save — invalid values are silently replaced with safe defaults rather than returning an error
- Cookies set with a 120-day expiry, path=/, and SameSite=Lax
- Preference resolution chain: PHP reads the cookie first, falls back to the preferences table in the database, then falls back to a hard-coded safe default — all three layers are implemented
- Active Preference Sources panel shows each preference's current value and whether it came from a cookie, a database default, or the built-in fallback
- Cookies Currently Set panel shows raw cookie names and values (or 'not set') for transparency
- Reset to Defaults button expires all four indiclex_* cookies and re-reads values from the preferences table
- New preference values are reflected immediately after the POST save, without a page redirect, so the user sees updated state right away
- Theme radio buttons in the form update their border color dynamically via JavaScript when the selection changes

Challenges & Decisions

- Reflecting the new preferences immediately after POST (rather than redirecting) avoids a round trip and prevents the user from briefly seeing stale values
- Empty string cookie values are treated as 'not set' and fall through to the database default — without this check, an empty cookie would block the fallback chain
- Storing system defaults in the preferences table (rather than hard-coding them everywhere) means an admin can change defaults in the database without a code deploy

Demo Notes

- Set results per page to 25, go to Search, run a query — confirm the larger page size is applied
- Set a default dictionary in preferences, go to Search — confirm it is pre-selected in the dropdown

- Click Reset to Defaults — confirm cookies are cleared and the source column shows 'db default'
- Clear browser cookies manually and reload the preferences page — confirm database defaults are shown as the source

Iteration 6 — Admin Authentication & Dashboard

Status: Complete

What Was Planned

- Build admin login with secure password hashing and session management
- Create the Admin Dashboard with live summary statistics
- Add role-based route protection to all admin pages

What Was Built

- Admin login form built with username and password fields — passwords stored as bcrypt hashes using PHP `password_hash()`
- Session management uses PHP sessions; `session_regenerate_id()` is called immediately after a successful login to prevent session fixation attacks
- Admin guard function included at the top of every admin page — unauthenticated requests are redirected to the login page with no content shown
- Admin Dashboard displays total dictionary count, total word entry count, per-dictionary entry counts, and active language breakdown — all queried live from the database
- Dashboard statistics update automatically whenever dictionaries or entries are added or removed
- users table supports role ENUM (admin/visitor) to allow for future expansion of access levels
- Schema seed includes a placeholder admin account with a bcrypt hash that must be replaced before production deployment
- bcrypt chosen as the hashing algorithm for OWASP compliance

Challenges & Decisions

- `session_regenerate_id()` is called right after successful authentication — this is a standard security measure that costs nothing and prevents a known attack vector
- Live queries for dashboard stats rather than cached values mean the numbers always reflect the current database state, which is important when entries are being actively imported
- The role column in the users table was included from the start even though only admin is used now, to avoid a schema migration later if visitor accounts are added

Demo Notes

- Attempt to navigate to an admin page directly without logging in — confirm redirect to the login page
- Log in with valid credentials and confirm dashboard statistics match the database entry counts
- Import a batch of entries, then return to the dashboard — confirm the counts have updated

Iteration 7 — Dictionary & Entry CRUD

Status: Complete

What Was Planned

- Build full CRUD for dictionaries on the catalog page
- Build full CRUD for individual word entries
- Implement confirmation before destructive operations

What Was Built

- Create Dictionary form on catalog.php lets users select source and target languages from a known-languages list supporting 13 Indic and common languages
- Dictionary name is auto-generated as 'Source – Target' (e.g. Telugu – English) — enforced in PHP rather than free-text entry to ensure consistent naming
- Duplicate dictionary check runs before insert — a clear error message is shown if a dictionary with that name already exists
- Delete Entry section uses a two-step form: select a dictionary, the page resubmits to load that dictionary's entries into a second dropdown, then the user selects and deletes the entry
- Delete confirmation uses a JavaScript confirm() dialog to prevent accidental deletions
- ON DELETE CASCADE on the dictionary_entries foreign key means deleting a dictionary automatically removes all its entries with no extra application code
- All database operations use prepared statements — no SQL string concatenation anywhere in the CRUD flows
- Catalog stats summary (active dictionary count, source language count) recalculates on every page load and reflects the latest state after any CRUD operation
- Language name, native script, and script name stored in the known_languages array in catalog.php — displayed as styled badges in the catalog table

Challenges & Decisions

- Auto-generating the dictionary name from language dropdowns prevents inconsistent casing or formatting that would make the catalog table harder to read
- The two-step entry delete flow (select dict first, then entry) avoids loading potentially thousands of entries into a single dropdown on the initial page load
- Cascade delete at the database layer is simpler and more reliable than writing a separate DELETE query for entries before deleting the dictionary

Demo Notes

- Create a new Telugu – Sanskrit dictionary — confirm it appears in the catalog table immediately
- Select that dictionary in the delete section and confirm its entries load into the second dropdown
- Delete an entry with the confirmation dialog — confirm it no longer appears in the dropdown afterward
- Delete a full dictionary — confirm its entries are also removed (cascade verified via phpMyAdmin)

Iteration 8 — REST API & Advanced Search

Status: Complete

What Was Planned

- Build the REST API endpoint GET /api/search with JSON responses
- Implement proper HTTP status codes for all response cases
- Add jQuery autocomplete to the search page
- Build the Word Length Matcher feature
- Configure .htaccess for clean URL routing

What Was Built

- REST API implemented in `api_search.php` — accepts GET requests with parameters: `query` (required), `dict_id`, `mode`, `page`
- All four search modes supported in the API using identical query logic to the main search page
- HTTP status codes: 200 for success, 400 for missing or invalid parameters, 404 for a non-existent dictionary, 405 for non-GET requests, 500 for database errors
- JSON response includes metadata: `query`, `dict_id`, `mode`, `page`, `per_page`, `total`, `total_pages`, `count`, and the results array
- Empty optional fields (`lang_3`, `pronunciation`, `part_of_speech`) are omitted from results to keep the JSON compact
- CORS headers set (`Access-Control-Allow-Origin: *`) with OPTIONS preflight handling for cross-origin API use
- Autocomplete on `search.php` fires after 2+ characters typed, calls `/api/search` in prefix mode via jQuery AJAX, and shows up to 8 suggestions in a dropdown
- Autocomplete supports keyboard navigation: arrow keys to move, Enter to select, Escape to close; closes on outside click
- Word Length Matcher: user enters a pattern using `?` as a wildcard for any single character — e.g. `???` matches all 3-letter words
- Quick-length badges (2–12) auto-fill the pattern with that many `?` characters and trigger an immediate match
- WLM results table shows headword, character length, translation, part of speech, dictionary name, and a Puzzle link to telugupuzzles.com pre-filled with the word
- Character counting in WLM uses `Array.from()` to correctly count Unicode graphemes for multi-byte Indic script characters
- `.htaccess` RewriteRule maps `/api/search` to `api_search.php`; sensitive files (`dbcon.php`, `.sql`, `.log`, `.env`) are blocked from direct access

Challenges & Decisions

- MIME escaping in the API mirrors the main search page exactly — `%` is added by PHP, not passed in from user input, to prevent wildcard injection through the API
- Autocomplete uses prefix mode specifically because it is the fastest query type for partial-word suggestions and returns the most natural results as someone types
- `Array.from()` for character counting was necessary because standard JavaScript `.length` counts UTF-16 code units, which gives wrong results for Telugu and other multi-byte scripts

- CORS enabled on the API so it can be called from the autocomplete JavaScript on the same server and from any external tool without browser security errors

Demo Notes

- Call `/api/search?query=love&mode=prefix` in the browser — verify JSON response with results array and correct metadata
- Call with no query parameter — verify 400 response with descriptive error message
- Type in the search box and confirm the autocomplete dropdown appears and is keyboard-navigable
- Click the '3' quick-length badge — confirm Word Length Matcher returns only 3-character headwords
- Click a Puzzle link — confirm it opens telugupuzzles.com with the correct word pre-filled

Iteration 9 — Comparison, Data Integrity & Reporting

Status: Complete

What Was Planned

- Build dictionary comparison and data integrity tools
- Implement reporting with live statistics
- Verify all integrity constraints are enforced end-to-end

What Was Built

- Dictionary Catalog stats panel shows active dictionary count and unique source language count, calculated live on every page load
- Data integrity enforced at multiple layers: UNIQUE constraint on (dict_id, lang_1) prevents duplicate headwords; ON DELETE CASCADE maintains referential integrity; triggers keep entry_count in sync
- vw_dictionary_stats view provides per-dictionary statistics (entry count, type, languages, created date) for reporting use
- vw_language_stats view aggregates word counts by language across all active dictionaries for the language distribution report
- Admin Dashboard (Iteration 6) serves as the primary reporting surface: total dictionaries, total entries, per-dictionary breakdowns, and language distribution
- Duplicate detection is handled at the database level — any insert that violates the UNIQUE constraint is caught, reported to the user, and skipped without aborting the rest of the operation
- Cross-dictionary comparison is available via the Catalog table, which shows all active dictionaries side by side with their language pairs, type, and entry counts
- All reporting queries use the live database so numbers always reflect the current state without any caching inconsistency

Challenges & Decisions

- Reporting views were built into the schema from the start (Iteration 1) rather than added later, so they are available to any page that needs statistics without duplicating GROUP BY logic
- Enforcing data integrity at the database level rather than only in application code means the constraints hold even if entries are modified directly in phpMyAdmin or through the API

Demo Notes

- View the Catalog stats panel and confirm the dictionary count and source language count are accurate
- Attempt to upload a file with a duplicate entry — confirm it is caught and skipped with a clear message
- Query vw_language_stats directly in phpMyAdmin and confirm the language-wise entry counts match the catalog
- Add a new dictionary and entries, then check the Admin Dashboard — confirm counts update immediately

Iteration 10 — Stabilization, Polish & Final Demonstration

Status: Complete

What Was Planned

- Freeze new feature development and conduct end-to-end testing
- Harden security and fix all identified bugs
- Polish the UI for consistency and responsiveness
- Finalize documentation and prepare the live demonstration

What Was Built

- .htaccess fully hardened: security headers (X-Content-Type-Options, X-Frame-Options, Referrer-Policy), gzip compression, browser caching rules for CSS/JS/images, no-cache on API responses, and explicit blocks on .sql, .log, .env, .sh, .bak files
- Options -Indexes prevents directory listing on Bluehost
- All form outputs wrapped in htmlspecialchars() to prevent XSS; all queries use prepared statements throughout the application
- Upload validation covers PHP upload error codes, extension check, MIME type check, and file size limit — all error cases show user-friendly messages
- End-to-end test pass completed across all pages and features: home upload, catalog create/delete, search all 4 modes, preferences save/reset, API endpoint, autocomplete, Word Length Matcher, admin login/dashboard
- Unicode rendering verified for Telugu and Sanskrit text in both light and dark themes across Chrome and Firefox
- UI consistency reviewed — sidebar, breadcrumbs, card headers, and button styles are uniform across all pages
- Mobile sidebar collapse tested on narrow viewports
- GitHub repository cleaned: .gitignore updated to exclude vendor/, uploads/, and sensitive config; schema SQL file finalized; README added
- Final demonstration rehearsed and delivered from the live Bluehost URL

Challenges & Decisions

- Upload modal auto-reopen on validation error was tested carefully — confirmed the modal stays open and shows the error rather than closing and losing user context
- Verified that SameSite=Lax on cookies works correctly on Bluehost's Apache environment without requiring HTTPS
- Confirmed the .htaccess RewriteRule for /api/search resolves correctly on Bluehost with mod_rewrite enabled — tested both with and without a trailing slash
- Unicode character length counting in the Word Length Matcher was verified with Telugu script words to confirm Array.from() gives the correct grapheme count

Demo Notes

- Full live demonstration from the Bluehost URL: Home → upload an .xlsx file → Catalog → create a dictionary → Search with all four modes → Preferences save and reset → call

/api/search in the browser → autocomplete demo → Word Length Matcher with quick badges
→ dark mode toggle

- All features performed correctly in the live demo with no errors or broken pages
- GitHub repository link shared with instructor showing clean commit history

Appendix

A. REST API Reference

The application exposes one search endpoint, accessible via a clean URL configured in .htaccess.

Method	GET
Clean URL	/indiclex/api/search
File	api_search.php
query (required)	Search term — max 200 characters
dict_id	Optional — restrict to a specific dictionary ID; omit to search all
mode	Optional — exact prefix suffix substring (default: exact)
page	Optional — page number for pagination (default: 1)

HTTP status codes:

- 200 — Success. Results array returned (may be empty).
- 400 — Missing or invalid parameter (e.g. no query, bad dict_id, unknown mode).
- 404 — dict_id not found or inactive.
- 405 — Non-GET method used.
- 500 — Database error.

B. Search Mode Reference

Mode	Description	Example Input	SQL Pattern
exact	Full headword match only	vanamu	vanamu
prefix	Words starting with the query	van	van%
suffix	Words ending with the query	amu	%amu
substring	Query appears anywhere in the word	ana	%ana%

C. Supported Languages

The following 13 languages are available for dictionary creation in catalog.php:

Language	Native Script	Language	Native Script
----------	---------------	----------	---------------

Telugu		Sanskrit	
Hindi	हिन्दी	English	English
Tamil	தமிழ்	Kannada	ಕನ್ನಡ
Malayalam	മലയാളം	Bengali	বাংলা
Marathi	मराठी	Gujarati	ગુજરાતી
Punjabi	ਪੰਜਾਬੀ	Odia	ଓଡ଼ିଆ
Urdu	اردو		